

Documentazione

Nel progetto si utilizzano socket TCP per lo scambio di messaggi tra lavagna e utenti e tra i vari utenti, questo per garantire l'affidabilità nello scambio dei messaggi che altrimenti sarebbe dovuta essere implementata tramite ulteriori controlli nel codice, e evitare quindi eventuali deadlock o errori causati da possibili ritardi, mancate consegne o consegne fuori ordine dei messaggi, che comprometterebbero quindi il corretto funzionamento del sistema.

I messaggi sono definiti in un formato del tipo:

COMANDO|PORTA Sorgente|PAYLOAD

I messaggi sono quindi strutturati in tre campi delimitati dal carattere "|", per il corretto scambio dei messaggi si assume che i campi non contengano il carattere delimitatore scelto, considerando anche che il tipo di informazioni che devono essere inviate difficilmente conterrà questo carattere.

- Il campo **COMANDO** contiene il comando da inviare, specificato come stringa secondo il formato definito nella descrizione dei comandi nella descrizione.
- Il campo **PORTA Sorgente** contiene il numero di porta della lavagna o dell'utente che invia il messaggio.
- Il campo **PAYLOAD** è variabile in base al comando specificato, può essere vuoto o contenere diverse informazioni in base al comando, le informazioni aggiuntive nel payload sono separate dal carattere "-", si assume anche in questo caso che le stringhe che compongono i vari campi non contengano il carattere delimitatore, in particolare i campi contenenti la descrizione delle attività delle card (sono inserite dall'utente).

I messaggi vengono preparati per l'invio dalla funzione `codifica_messaggio()` che prepara il buffer per l'invio e le informazioni vengono ottenute tramite la funzione `decodifica_messaggio()` che estrapola le informazioni separate dai delimitatori e le salva in apposite variabili.

Ho scelto di utilizzare questo formato per la flessibilità del campo payload, che permette in modo semplice di inserire informazioni per eventuali nuovi comandi, e per la semplicità nella lettura dei campi, la cui lettura termina sempre in corrispondenza del delimitatore.

Questo formato limita ovviamente le informazioni che si possono inserire nei vari campi, che non possono contenere i caratteri delimitatori, e quindi il sistema è potenzialmente soggetto a errori di parsing.

Il payload dei messaggi è definito nel seguente modo in base al comando:

- **HELLO, QUIT, MOVE_CARD, SHOW_LAVAGNA, PING_USER, PONG_LAVAGNA, REQUEST_USER_LIST** : campo vuoto
- **CREATE_CARD** : TESTO_ATTIVITA
- **SEND_USER_LIST** : NUM_UTENTI-LISTA_UTENTI
- **HANDLE_CARD** : ID_CARD-TESTO_ATTIVITA-NUM_UTENTI-LISTA_UTENTI
- **ACK_CARD** : ID_CARD
- **REVIEW_CARD** : GIUDIZIO
- **CARD_DONE** : ID_CARD

Nei messaggi in cui si invia la lista degli utenti e il numero di utenti la lista non include mai l'utente a cui il messaggio viene inviato mentre il numero di utenti è il numero totale di utenti attualmente registrati alla lavagna quindi lo include. La lista degli utenti è formata dai numeri di porta degli utenti separati dal carattere ",".

Il campo giudizio del messaggio **REVIEW_CARD** vale 0 per le richieste di revisione e 1 per i giudizi inviati, gli utenti utilizzano quindi lo stesso comando quando vogliono richiedere una revisione e quando devono inviare un giudizio a un altro utente, con la differenza del campo giudizio che ne specifica il tipo, questo per evitare di definire un altro comando e limitarmi ai comandi specificati nella descrizione del progetto. Nei messaggi **REVIEW_CARD** non ho inserito ulteriori campi data l'assunzione che gli utenti inviano sempre giudizi positivi, quindi un qualsiasi **REVIEW_CARD** con campo giudizio a 1 corrisponderà a un giudizio positivo.

Per i messaggi con campo payload vuoto non è necessario specificare alte informazioni, perchè vengono prelevate in altro modo o perchè sono sufficienti i campi comando e porta sorgente.

La lavagna è definita come un vettore di card, per semplicità può contenere un numero massimo di card scelto in base a stime sul possibile utilizzo pratico del sistema, anche il numero massimo di utenti è scelto

in base a questo criterio. Le card sono definite come struct contenenti id della card, colonna i cui è inserita, testo dell'attività, utente a cui è assegnata e timestamp dell'ultima modifica.

Le card sono inserite nella lavagna senza distinzioni o ordinamenti in base alla colonna di appartenenza, questo per semplificare lo spostamento delle card che in questo modo consiste semplicemente nella modifica del campo "colonna" della card da spostare.

Nella visualizzazione della lavagna le descrizioni sono troncate per semplicità nella formattazione delle card, il testo completo è comunque presente nelle card.

Quando un utente si collega viene inizialmente inserito in un vettore temporaneo in attesa della ricezione del messaggio HELLO per la registrazione, alla ricezione della richiesta di registrazione gli utenti vengono inseriti nella lista di utenti registrati. Nella lista di attesa gli utenti sono rappresentati solo dal socket che gli viene associato, quando la lavagna riceve HELLO riceve anche il numero di porta e quindi nella lista degli utenti registrati per ogni utente si specificherà il socket, la porta e la card associata all'utente in quel momento. La lista degli utenti è mantenuta ordinata e senza buchi in modo da poterla scorrere fino al numero di utenti attualmente connessi.

Gli utenti sono mantenuti ordinati nella lista in base al numero di porta per facilitare l'invio dei messaggi da parte della lavagna

La select nella lavagna ha un timeout per evitare che la lavagna si blocchi nelle fasi di scambio dei messaggi PING_USER e PONG_LAVAGNA.

Gli utenti memorizzano le informazioni sulla card a loro associata nella struttura card_gestita, dove vengono salvati l'id della card (0 se non è associata) e il testo dell'attività.

Ogni utente ha inoltre una lista degli utenti connessi alla lavagna (escluso se stesso) aggiornata all'ultima ricezione di SEND_USER_LIST, una lista dei socket degli altri utenti per la revisione e una lista degli utenti a cui deve inviare l'approvazione (utenti che hanno richiesto la revisione). Gli utenti hanno anche una variabile in cui tengono conto delle approvazioni ricevute fino a quel momento quando richiedono la revisione per una card.

Ogni utente attende quindi messaggi dalla lavagna, dagli utenti che si connettono per inviare richieste di revisione e dagli utenti che devono inviargli le approvazioni.

Sia nella lavagna che negli utenti il controllo dei socket è gestito tramite select() per semplicità nella gestione data anche l'organizzazione in liste dei socket dei vari utenti.

Nel caso in cui quando un utente al termine dello svolgimento della sua card non ha altri utenti a cui richiedere la revisione, e quindi è l'unico utente connesso alla lavagna, l'utente attende un tempo fissato e richiede nuovamente la lista degli utenti connessi fino a che non se ne connette almeno un altro. Dato che quando l'utente richiede la lista degli utenti ha già terminato l'attività e ne attende solo l'approvazione, ho scelto di attendere una nuova connessione di un utente alla lavagna piuttosto che l'abbandono della card, che ne richiederebbe di nuovo lo svolgimento nonostante sia stata già terminata una prima volta.

Nel caso in cui invece un utente richieda la revisione agli utenti che ha nella lista ma nel frattempo uno di questi si disconnette, l'utente rimarrebbe in attesa di approvazione a vuoto, per questo alla disconnessione di un utente che era nella lista considero un giudizio in meno tra quelli attesi, in questo modo la fase di revisione può terminare ugualmente. Anche in questo caso non ho scelto di reinserire la card in "to do" perché alla richiesta della lista l'attività è già terminata e altrimenti richiederebbe di nuovo lo svolgimento. Con questa scelta si evita inoltre di ritrasmettere la lista degli utenti connessi, necessaria invece nel caso precedente dell'unico utente connesso a causa della sleep();

Per la compilazione usare i comandi:

```
gcc -Wall lavagna.c -o lavagna
gcc -Wall utente.c -o utente
```

Per l'esecuzione come da specifiche:

```
./lavagna
./utente <porta>
```